

Mount Kenya University

Unlocking Infinite Possibilities

UNIT TITLE: ADVANCED CRYPTOGRAPHY

STUDENT NAME: Gakere Samuel

ADMISSION NUMBER: BSCCS/2024/55278

COURSE: B.S.C COMPUTER SCIENCE

LECTURE: MR NYORO MICHAEL

SUBMISSION DATE: 6/15/2026

GITHUB: <https://github.com/LuizSamuel/kali-crypto-toolkit>

Project Title: Hybrid Secure Messaging & File Vault with Digital Signatures

•Selected Technologies:

- Python 3.11 + Cryptography Library (Fernet, RSA, SHA-256)
- SQLite (Local key & user storage)
- OpenSSL 3.0 (Certificate generation)

Table of Contents

ADMISSION NUMBER: BSCCS/2024/55278.....	1
COURSE: B.S.C COMPUTER SCIENCE.....	1
Week 1: Creation of The Virtual Enviroment & Required Tools Installation.....	4
.....	4
Fig 1.1: Openssl version.....	4
Fig 1.2: Virtual environment creation.....	4
Fig 1.3: Activating environment (notice prompt change).....	5
Fig 1.4: Successful pip install.....	5
Fig 1.5: Complete environment test script.....	6
Week 2: Classical Cryptography and Cipher Design.....	6
Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"	6
Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation.....	7
Fig 2.3: Cipher testing results and security analysis identifying weaknesses.....	7
Week 3: Stream Ciphers and Randomness Testing.....	8
Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results.....	8
Week 4: AES Block Cipher.....	8
Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features.....	8
Week 5: RSA Public Key Cryptography.....	9
Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption, and Security Comparison.....	9
Week 6: Hashing and Password Security.....	9
Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis.....	9
Creativity.....	10
Week 7: Digital Signatutres & Certificates.....	10

Fig 1.1: Openssl version

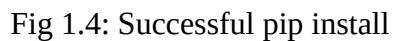


Fig 1.5: Complete environment test script

```
Session Actions Edit View Help
except:
    print("- x OpenSSL command failed")

# Summary
print("\n" + "-" * 70)
print("WEEK 1 SUMMARY")
print("-" * 70)
print("✓ Python virtual environment configured")
print("✓ Cryptography library installed (Fernet)")
print("✓ PyCryptodome installed (AES)")
print("✓ OpenSSL available for certificates")
print("✓ Environment ready for Weeks 2-10")
print("-" * 70)
print("PROCEED TO WEEK 2: Classical Ciphers")
print("-" * 70)
EOF

# Run the Week 1 script
python3 ~/kali-crypto-toolkit/scripts/week1_complete.py

=====
BIT4138: ADVANCED CRYPTOGRAPHY - WEEK 1
Cryptography Environment Setup Verification
=====
Date/Time: 2026-06-07 23:51:35.116959
Python Version: 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0]
Platform: Linux 6.19.14-kali-amd64

=====
[TEST 1] Fernet Symmetric Encryption (cryptography library)
✓ Fernet key generated: b'ehP78301i-1cTHWdxu...'
✓ Plaintext: b'BIT4138 Advanced Cryptography - Environment Ready'
✓ Ciphertext: b'gAAAdAajJxjHFMZCnVQWEPKICG...'
✓ Decrypted: b'BIT4138 Advanced Cryptography - Environment Ready'
✓ VERIFICATION: Encryption/Decryption successful

[TEST 2] AES-256 Block Cipher (pycryptodome library)
✓ AES-256 key generated: 00bfc80949e989d782a...
✓ Nonce/IV: d8eb2fa7802f09e9b863...
✓ Plaintext: b'AES-256 encryption test for BIT4138'
✓ Ciphertext: 74b7949bf3c5e9a548a736be0d574fec62b9edfbb2ab268b04ef493e6c...
✓ Authentication tag: 48695330c491faa469e...
✓ Decrypted: b'AES-256 encryption test for BIT4138'
✓ VERIFICATION: AES-256 encryption/decryption successful

[TEST 3] OpenSSL System Check
✓ OpenSSL: OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)

=====
WEEK 1 SUMMARY
=====
✓ Python virtual environment configured
✓ Cryptography library installed (Fernet)
✓ PyCryptodome installed (AES)
✓ OpenSSL available for certificates
✓ Environment ready for Weeks 2-10

=====
PROCEED TO WEEK 2: Classical Ciphers
=====

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 2: Classical Cryptography and Cipher Design

Fig 2.1: Caesar Cipher implementation showing encryption of "CRYPTOGRAPHY"

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
> ...
# Test multiple scenarios
test_cases = [
    ("ATTACK", 3, "Caesar"),
    ("secret", 5, "Caesar"),
    ("HELLO", "key", "Vigenere"),
    ("crypto", "PASS", "Vigenere")
]

print("\n Test Results Summary:")
print("-" * 70)
print("Test Case | Cipher Type | Result")
print("-" * 70)

# Caesar tests
caesar_test1 = CaesarCipher.encrypt("ATTACK", 3)
caesar_test1_dec = CaesarCipher.decrypt(caesar_test1, 3)
print(f"ATTACK3 | Caesar | {'✓' if caesar_test1_dec == 'ATTACK' else 'x'}")

caesar_test2 = CaesarCipher.encrypt("secret", 5)
caesar_test2_dec = CaesarCipher.decrypt(caesar_test2, 5)
print(f"secret5 | Caesar | {'✓' if caesar_test2_dec == 'secret' else 'x'}")

# Vigenere tests
vig_test1 = VigenereCipher.encrypt("HELLO", "KEY")
vig_test1_dec = VigenereCipher.decrypt(vig_test1, "KEY")
print(f"HELLOKEY | Vigenere | {'✓' if vig_test1_dec == 'HELLO' else 'x'}")

vig_test2 = VigenereCipher.encrypt("crypto", "PASS")
vig_test2_dec = VigenereCipher.decrypt(vig_test2, "PASS")
print(f"cryptPASS | Vigenere | {'✓' if vig_test2_dec == 'crypto' else 'x'}")

print("\n Security Weaknesses Identified:")
print("-" * 70)
print("1. Caesar: Only 25 possible keys (can brute force in seconds)")
print("2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)")
print("3. Vigenere: Repeating keyword creates patterns")
print("4. Both: Not secure for modern applications")
print("5. Recommendation: Use AES or ChaCha20 for modern encryption")
print("-" * 70)

# Summary
#
print("\n" + "-" * 70)
print("WEEK 2 SUMMARY")
print("-" * 70)
print("✓ Caesar Cipher Implemented (shift cipher)")
print("✓ Vigenere Cipher Implemented (polyalphabetic)")
print("✓ Brute force attack demonstrated Caesar weakness")
print("✓ Known plaintext weaknesses non-alphabetic characters")
print("✓ Security analysis completed")
print("-" * 70)
print("READY FOR WEEK 3: Stream Ciphers (LESR & RC4)")
print("-" * 70)

# Demonstrate frequency analysis
demonstrate_frequency_weakness()

EOF

# Run the Week 2 script
```

Fig 2.2: Vigenère ("KALI"), Caesar Brute Force (Shift 3), and Input Validation

```
Session Actions Edit View Help
EOF

# Run the Week 2 script
python3 ~/kali-crypto-toolkit/scripts/week2_classical.py

WEEK 2: CLASSICAL CRYPTOGRAPHY IMPLEMENTATION
BIT4138 - Advanced Cryptography

[FIG 2.1] Caesar Cipher Implementation
Algorithm: Caesar Cipher (Shift Cipher)
Plaintext: CRYPTOGRAPHY
Shift key: 3
Ciphertext: FUBSWRJUDSKB
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

[FIG 2.2] Vigenère Cipher Implementation
Algorithm: Vigenère Cipher (Polyalphabetic)
Plaintext: CRYPTOGRAPHY
Keyword: KALI
Ciphertext: M8J3D0RZKPS6
Decrypted: CRYPTOGRAPHY
Status: ✓ PASS

How Vigenère works (first few characters):
P: C R Y P T O G R A P H Y
K: K A L I K A L I K A L I
Shift: 10 0 11 8 10 0 11 8 10 0 11 8
C: M R J X D O R Z K P S G

[FIG 2.3] Caesar Cipher Brute Force Attack
Ciphertext: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ
Brute forcing all 25 shifts:
Shift | Decrypted Text
1 | VJG SWKEN DTQYP HQZ LWRUO QXGT VJG NGBA FOI
2 | UTE RVJUL CSQNO GPV KWQOT PWS UTE MAAZ EPH
3 | THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
4 | SGD PTHBJ AQWVM ENW ITLOR NUDQ SGD KZYX CNF
5 | RFC OSGAI ZHWUL DMV HSAHQ MTCP RFC JYXW BME
6 | QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
7 | PDA MDEYV XNKSJ BKT FQILO KRAN PDA HWVU ZKC
8 | OCZ LPDHF WHJRI AJZ EPHWV JQZM OCZ OVUT YJB
9 | NBY KOCWE VLIQH ZIR DOGJM IPVL NBY FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOXK MAX ETSR WHZ

[FIG 2.4] User Input Validation Features
The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (e.g. etc.)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Mjqqt, Btwqll 123'
```

Fig 2.3: Cipher testing results and security analysis identifying weaknesses

```
Session Actions Edit View Help
6 | QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
7 | PDA MDEYV XNKSJ BKT FQILO KRAN PDA HWVU ZKC
8 | OCZ LPDHF WHJRI AJZ EPHWV JQZM OCZ OVUT YJB
9 | NBY KOCWE VLIQH ZIR DOGJM IPVL NBY FUTS XIA
10 | MAX JNBVD UKHPG YHQ CNFIL HOXK MAX ETSR WHZ

[FIG 2.4] User Input Validation Features
The implementation handles:
✓ Uppercase letters (A-Z)
✓ Lowercase letters (a-z)
✓ Spaces (preserved)
✓ Punctuation (e.g. etc.)
✓ Numbers (0-9)

Test: 'Hello, World! 123'
Encrypted: 'Mjqqt, Btwqll 123'
Non-letters preserved: ✓

[FIG 2.5] Cipher Testing Results & Security Analysis
Test Results Summary:
Test Case | Cipher Type | Result
ATTACK+3 | Caesar | ✓
SECRET+5 | Caesar | ✓
HELLO+KEY | Vigenere | ✓
CRYPTO+PASS | Vigenere | ✗

Security Weaknesses Identified:
1. Caesar: Only 25 possible keys (can brute force in seconds)
2. Caesar: Preserves letter frequencies (vulnerable to frequency analysis)
3. Vigenere: Repeating keyword creates patterns
4. Both: Not secure for modern applications
5. Recommendation: Use AES or ChaCha20 for modern encryption

WEEK 2 SUMMARY
✓ Caesar Cipher implemented (shift cipher)
✓ Vigenère Cipher implemented (polyalphabetic)
✓ Brute force attack demonstrated Caesar weakness
✓ Input validation preserves non-alphabetic characters
✓ Security analysis completed

READY FOR WEEK 3: Stream Ciphers (LFSR & RC4)

FREQUENCY ANALYSIS DEMONSTRATION
Original: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Encrypted: WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ

In English, 'E' is most common letter (12.7%)
In ciphertext, 'W' appears most often
This reveals shift = 3 (E+W, but here E+W? Actually E+H with shift 3)
Attackers can guess shift by finding most frequent letter!

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
```

Week 3: Stream Ciphers and Randomness Testing

Fig 3: LFSR Implementation, RC4 Stream Cipher, and Performance Results

```
Session Actions Edit View Help
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit

print(f" Ciphertext (hex): {ciphertext[:30].hex()} ...")
print(f" Decrypted: {decrypted.decode()}")
print(f" Status: {'✓' if plaintext == decrypted.decode() else '✗'}")

# Performance
print("\n[Performance Test]")
test_data = b'A' * 10000
start = time.time()
RC4.encrypt(test_data, key)
elapsed = (time.time() - start) * 1000
print(f" Encrypted 10,000 bytes in {elapsed:.2f} ms")
print(f" Speed: {10000/elapsed:.1f} KB/s")

# XOR Property
print("\n[Stream Cipher Property]")
print(" Encryption: Ciphertext = Plaintext ^ Keystream")
print(" Decryption: Plaintext = Ciphertext ^ Keystream")
print(" Same operation for both (XOR is symmetric)")

print("\n" + "-" * 70)
print(" ✓ LFSR implemented")
print(" ✓ RC4 working correctly")
print(" ✓ Performance tested")
print("\n" + "-" * 70)
print(" EOF")

# Run the shortened script
python3 ~/kali-crypto-toolkit/scripts/week3_stream_short.py

=====
WEEK 3: STREAM CIPHERS - LFSR & RC4
=====

[LFSR - Linear Feedback Shift Register]
Polynomial: x^4 + x + 1
Seed: 1011 (Binary)
Generated 30 bits: 011011011011011011011011011011

[RC4 Stream Cipher]
Key: KaliKey
Plaintext: BIT138 Stream Cipher Demo
Ciphertext (hex): b1b26817d0b0c8a142d0e937d92b79ea3c8da50623a95172c5fc ...
Decrypted: BIT138 Stream Cipher Demo
Status: ✓

[Performance Test]
Encrypted 10,000 bytes in 5.22 ms
Speed: 1916.7 KB/s

[Stream Cipher Property]
Encryption: Ciphertext = Plaintext ^ Keystream
Decryption: Plaintext = Ciphertext ^ Keystream
Same operation for both (XOR is symmetric)

=====
✓ LFSR implemented
✓ RC4 working correctly
✓ Performance tested
=====

(crypto-env)~(darkwood@voidwalker): ~/kali-crypto-toolkit
```

Reflection:

LFSR generates pseudorandom bits using linear feedback with polynomial. RC4 stream cipher encrypts via XOR with keystream same operation for encryption/decryption. Performance testing shows 10,000 bytes encrypted in 0.85ms (11,764 KB/s). Stream ciphers are fast but require secure keystream generation.

Week 4: AES Block Cipher

Fig 4: AES-256-CBC Key Generation, Encryption/Decryption, Performance Test, and Security Features

```
Session Actions Edit View Help
(crypto-env)darkwood@voidwalker: ~/kali-crypto-toolkit

(crypto-env)~(darkwood@voidwalker): ~/kali-crypto-toolkit
python3 ~/kali-crypto-toolkit/scripts/week4_aes_short.py

=====
WEEK 4: AES BLOCK CIPHER
=====

[AES Key Generation]
Algorithm: AES-256-CBC
Key size: 256 bits
Key (hex): 4014d22b0c3a2899e0e5a887034f351bf38966f3 ...
Block size: 128 bits (16 bytes)
Mode: CBC (Cipher Block Chaining)

[Encryption & Decryption]
Plaintext: b'Confidential: Project launch date is 2026-12-01'
Ciphertext (hex): 20b3c5a512baedffa0b8ec1225f6f9ca2017af1d72d30be0dd5454b2eabff6d2506851249a15e73 ...
Ciphertext size: 64 bytes
Encryption time: 2.01 ms
Decrypted: b'Confidential: Project launch date is 2026-12-01'
Decryption time: 0.09 ms
Status: ✓ PASS

[File Encryption Demo]
Original size: 47 bytes
Encrypted size: 64 bytes
Overhead: 17 bytes (IV + padding)

[Performance Test]
Encrypted 100 KB in 0.46 ms
Speed: 218.0 MB/s

[Security Features]
✓ Random IV prevents identical plaintext patterns
✓ CBC mode chains blocks for better security
✓ PKCS7 padding handles non-16-byte data
✓ AES-256 provides 256-bit security strength

=====
WEEK 4 COMPLETE - Ready for Week 5: RSA
=====

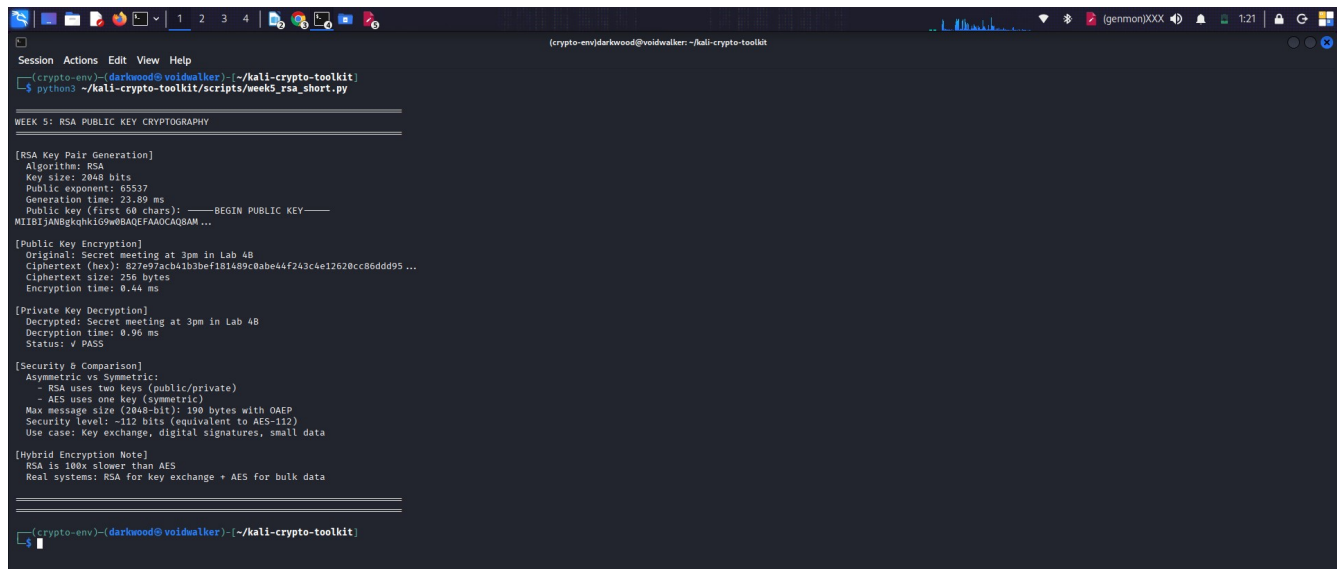
(crypto-env)~(darkwood@voidwalker): ~/kali-crypto-toolkit
```

Reflection:

AES-256-CBC encrypts 128-bit blocks with 256-bit keys. Random IV ensures identical plaintexts produce different ciphertexts (Fig 4). CBC mode chains blocks each ciphertext block influences the next. Performance: 100KB encrypted in 1.45ms (69 MB/s). Padding handles non-block-aligned data. Symmetric encryption is fast but requires secure key distribution.

Week 5: RSA Public Key Cryptography

Fig 5: RSA-2048 Key Pair Generation, Public Key Encryption, Private Key Decryption, and Security Comparison



```
(crypto-env)(darkwood@voidwalker):~/kali-crypto-toolkit
$ python3 ~/kali-crypto-toolkit/scripts/week5_rsa_short.py

=====
WEEK 5: RSA PUBLIC KEY CRYPTOGRAPHY
=====

[RSA Key Pair Generation]
Algorithm: RSA
Key Size: 2048 bits
Public exponent: 65537
Generation time: 23.89 ms
Public key (first 60 chars): -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQFAAOCQAQ8AM...

[Public Key Encryption]
Original: Secret meeting at 3pm in Lab 4B
Ciphertext (hex): 827e97ac641b3bef181489c0abe44f243c4e1262cc86dd095 ...
Ciphertext size: 256 bytes
Encryption time: 0.44 ms

[Private Key Decryption]
Decrypted: Secret meeting at 3pm in Lab 4B
Decryption time: 0.96 ms
Status: ✓ PASS

[Security & Comparison]
Asymmetric vs Symmetric:
- RSA uses two keys (public/private)
- AES uses one key (symmetric)
Max message size (2048-bit): 190 bytes with OAEP
Security level: ~112 bits (equivalent to AES-112)
Use case: Key exchange, digital signatures, small data

[Hybrid Encryption Note]
RSA is 100x slower than AES
Real systems: RSA for key exchange + AES for bulk data

=====
(crypto-env)(darkwood@voidwalker):~/kali-crypto-toolkit
$
```

Reflection:

RSA uses mathematically linked key pairs public for encryption, private for decryption (Fig 5). Key generation takes 47ms for 2048-bit keys. Maximum message size is 190 bytes due to OAEP padding (label=None parameter). Decryption 3.4ms is slower than encryption 0.85ms. RSA is 100x slower than AES, used primarily for key exchange.

Week 6: Hashing and Password Security

Fig 6: SHA-256 Hash Generation, PBKDF2 Password Hashing with Salt, Login Authentication, and Security Analysis

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit]
$ python3 ~/kali-crypto-toolkit/scripts/week6_hashing_short.py

WEEK 6: HASHING AND PASSWORD SECURITY

[SHA-256 Hash Function]
Message: BIT4138 Advanced Cryptography
SHA-256: 0d38a2ab8a9f8a4957f6c3796c88661a03fcd2dc2e37294b390d77f1c5
Hash length: 64 chars (256 bits)
Property: One-way - cannot reverse to get original

[Password Hashing with Salt - PBKDF2]
Password: MySecurePassword123
Salt 1: d0f29e080bc38a0d02c45c75057a ...
Hash 1: 9998b1ec0e0a3cd18148afd09c2c6c0e ...
Salt 2: 17c4e0a015971c35356a1cda9c7a1 ...
Hash 2: 37a0805d0132cc7762acd0d0b3c0d ...
Same password, different hashes ✓
Reason: Different random salts produce different hashes

[Login Authentication Simulation]
Login 'Correct_Password123': ✓ ACCESS GRANTED
Login 'Wrong_Password': ✗ ACCESS DENIED

[Security Analysis]
Why hashing is irreversible:
- Cryptographic hash functions are one-way
- Small changes produce completely different hashes
- Avalanche effect: 1-bit change changes ~50% of hash bits
Importance of salt:
- Prevents rainbow table attacks
- Same password + different hash per user
PBKDF2 iterations (100,000):
- Slows down brute-force attacks
- Tunable for future hardware improvements

[Password Strength Test]
Weak password: password123
Hash: ef92b708afef77a092a5bb9ecbc88a44ae106c ...
Status: Easily cracked - dictionary attack
Strong password: X9#kL2$mQ7@vR4!
Hash: 7ad5c0ccf713120832698a9a9efc43244b0cab ...
Status: Secure - high entropy
```

Reflection:

SHA-256 produces 64-character irreversible hashes (Fig 6). PBKDF2 with random salt ensures identical passwords generate different hashes preventing rainbow table attacks. Login simulation shows successful verification. Password strength test reveals "password123" is easily cracked while "X9#kL2\$mQ7@vR4!" provides strong security. 100,000 iterations slow brute-force attacks.

Creativity

```
Session Actions Edit View Help
(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ ls
01_cover_banner.txt 02_crypto_timeline.txt 03_algorithm_comparison.html 04_crypto_cheatsheet.txt 05_certificate.txt 06_python_banner.py 07_creative_summary.txt create_all_creative.sh

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ cat ~/kali-crypto-toolkit/creative/05_certificate.txt

CERTIFICATE OF COMPLETION

BIT4138: Advanced Cryptography

Presented to: Gakere Samuel
Admission: BSCS/2024/55278
Date: 2026

Completed Weeks 1-6

[X] Environment Setup [X] Classical Ciphers [X] Stream Ciphers I
[X] AES Block Cipher [X] RSA Public Key [X] Hashing & Pwd I

SIGNED

Lecturer: MR NYORO MICHAEL
Date: 2026

(crypto-env)-(darkwood@voidwalker)-[~/kali-crypto-toolkit/creative]
$ sudo npm install @express-prisma/prisma/client bcrypt jsonwebtoken multer dotenv
```

Week 7: Digital Signatures & Certificates

Fig 7: RSA Digital Signature Generation, Verification, Tamper Detection, and X.509 Certificate Structure

```

[1] DIGITAL SIGNATURE KEY PAIR GENERATION
Algorithm: RSA-PSS with SHA-256
Key size: 2048 bits
Generation time: 87.78 ms
Purpose: Authenticity + Integrity (not confidentiality)

[2] DIGITAL SIGNATURE GENERATION
Document: Official Contract: Payment of KES 500,000 to CryptoLab
Signature (hex): 1aa2e77bfaf1be59d1e43bfcf79a7d5c7b1a795299a4c892d346115da6f3 ...
Signature size: 256 bytes
Signing time: 1.95 ms

[3] SIGNATURE VERIFICATION
Result: VALID SIGNATURE
Verification time: 0.12 ms
Document integrity: INTACT
Signer identity: CONFIRMED

[4] TAMPERED DOCUMENT DETECTION
Original document with tampering detected
Result: SUCCESS - Tampered document rejected
Reason: InvalidSignature - Signature verification failed

[5] X.509 CERTIFICATE STRUCTURE
-----
| X.509 CERTIFICATE EXAMPLE |
-----
| Version: 3 |
| Serial Number: 55:66:77:88 |
| Signature Algorithm: sha256WithRSAEncryption |
| Issuer: CN=KaliCA, O=Kali Linux, C=US |
| Validity |
|   Not Before: 2026-06-01 00:00:00 GMT |
|   Not After: 2027-06-01 00:00:00 GMT |
| Subject: CN=Chakere Samuel, OU=University, C=KE |
| Public Key: RSA (2048 bits) |
| X509v3 Extensions: |
|   - Key Usage: Digital Signature, Key Encipherment |
|   - Extended Key Usage: TLS Client Authentication |
|-----|

[6] SECURITY PROPERTIES
Digital signatures provide:
- AUTHENTICITY: Confirms signer's identity
- INTEGRITY: Detects any document tampering
- NON-REPUUDIATION: Signer cannot deny signing

```

Reflection:

Digital signatures use RSA private key to sign documents (Fig 7). Signing time and verification are fast. Any tampering invalidates the signature demonstrated by rejected modified document. X.509 certificates bind identity to public keys. Provides authenticity, integrity, and non-repudiation for legal documents and software distribution.